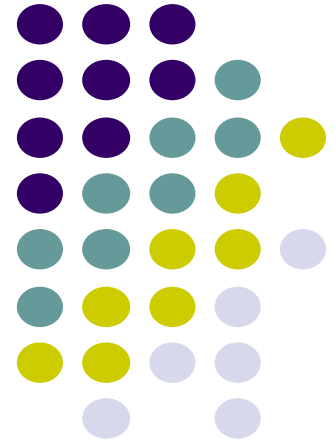


ErSE222: Machine learning in Geoscience

April. 22th, 2026

Tariq Alkhalifah and Omar Saad



Examples of generative modelling



- Can you imagine a **dog driving a car**?
- Can you imagine a **story of a young boy travelling to the moon**?
- Can you imagine a **soundtrack of a videogame**?
- Can you imagine a **panda surfing in the sea**?

*As humans, we have **great imagination** -> and sometimes can put it into practice...*

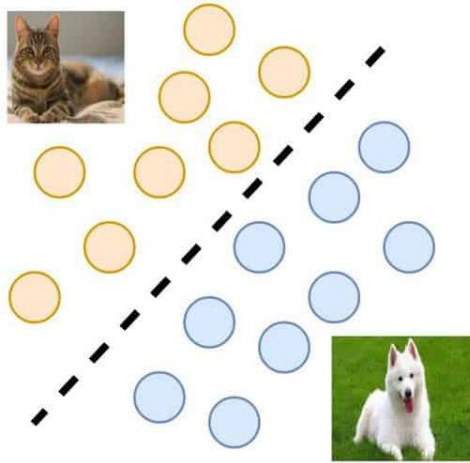
Generative Modelling: what and how?



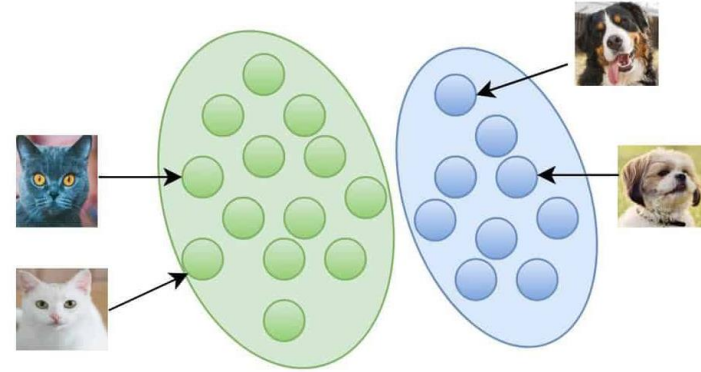
*Generative modelling is a subset of artificial intelligence that aims to **generate new and original content** (images, text, music, videos, etc.) using machine learning tools.*

- Some training data: $\mathbb{X} = \{\mathbf{x}^{(1)}, \mathbf{x}^{(2)}, \dots, \mathbf{x}^{(N_s)}\}$
- A model with learnable parameters (which learns the data): f_{θ}
- A procedure for the model to learn: $\mathbf{x}^{(i)} \rightarrow f_{\theta}$
- A procedure to generate data (or evaluate prob): $f_{\theta} \rightarrow \mathbf{x}^{(new)} / p(\mathbf{x}^{(new)})$

Generative Modelling: what and how?



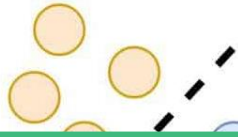
Discriminative



Generative

Source: <https://learnopencv.com/generative-and-discriminative-models/>

Generative Modelling: what and how?



$p(y|\mathbf{x})$ from $(\mathbf{x}^{(i)}, y^{(i)})$



Discriminative



$p(\mathbf{x})$ from $\mathbf{x}^{(i)}$
or $p(\mathbf{x}|y)$ from $(\mathbf{x}^{(i)}, y^{(i)})$

Generative

Generative Modelling: what and how?



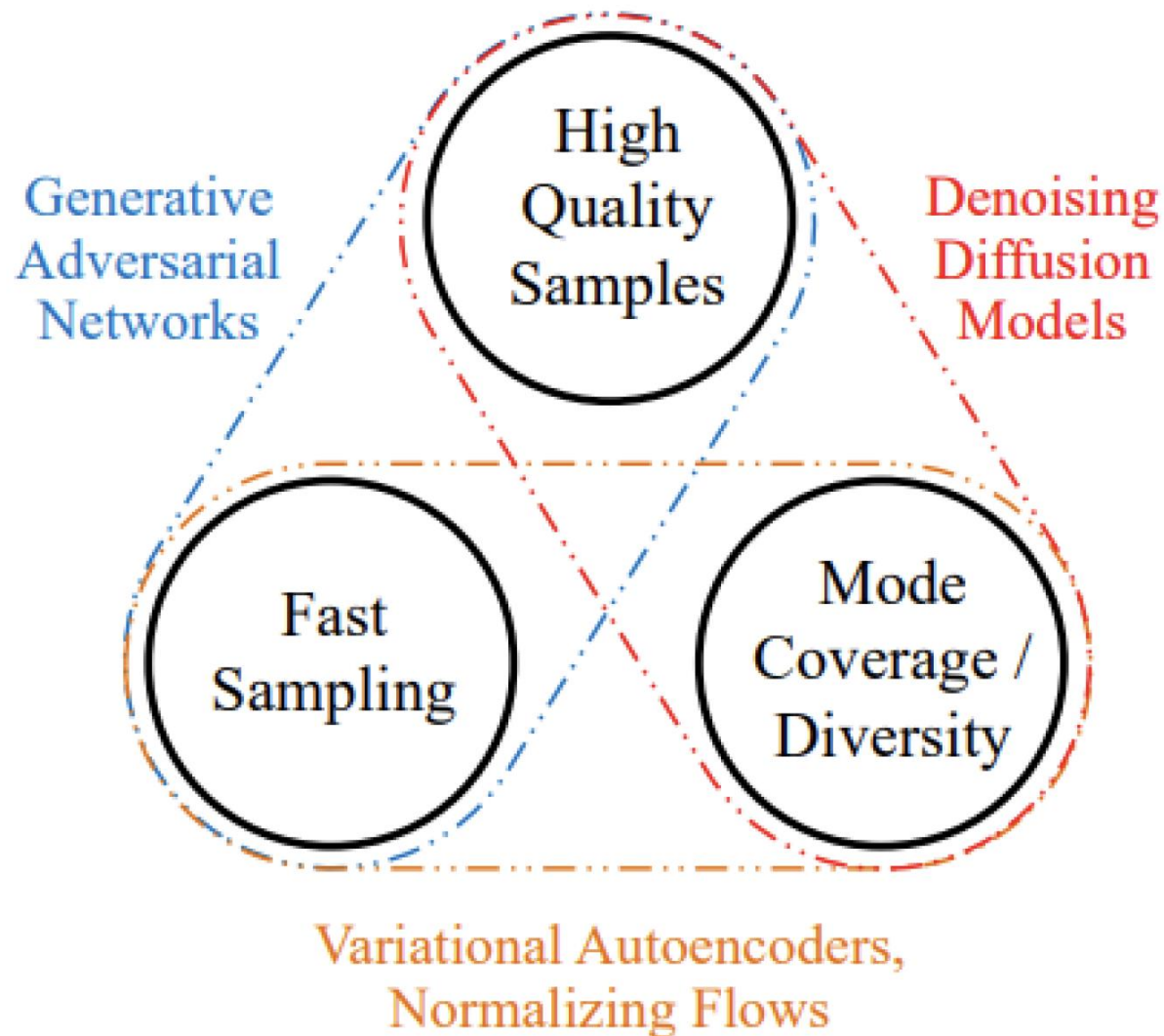
Ideally a generative modelling is a model solving the following problem

$$\operatorname{argmax}_{\theta} p(\mathbf{x}) \text{ from } \mathbf{x}^{(i)}$$

Maximum-likelihood estimator

We will see that this is not easy to do, usually approximations are required.

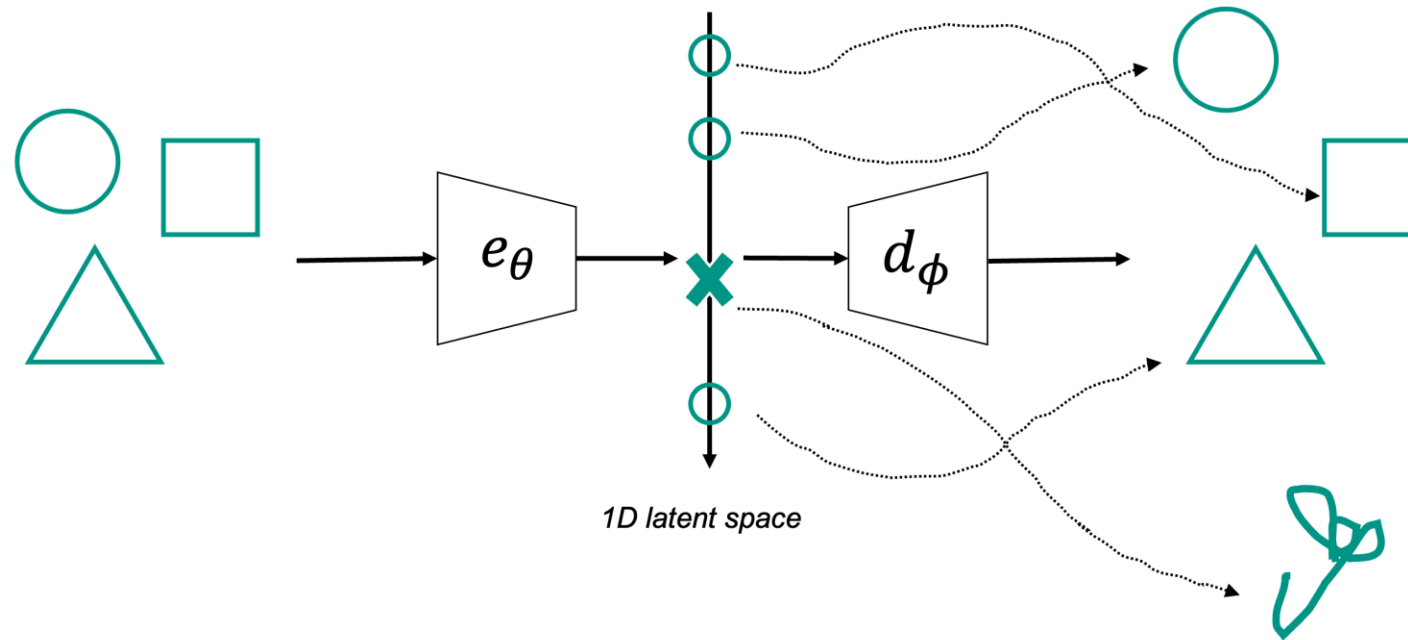
Generative Modelling: types of models



AutoEncoders as generative models



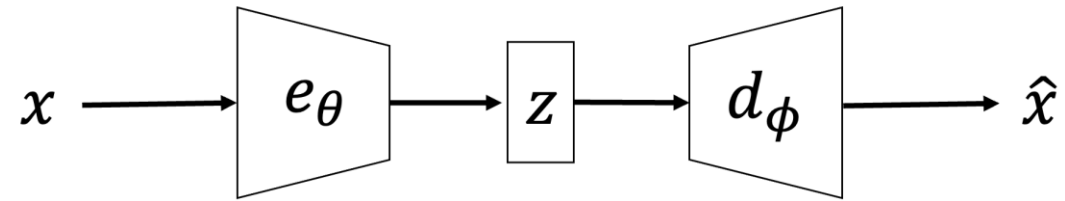
What if we **sample a new embedding vector** from z and then have the decoder reconstruct the “input” from it?



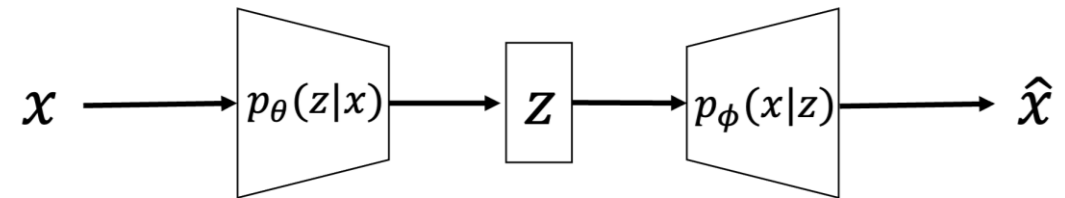
Variational AutoEncoders



AE

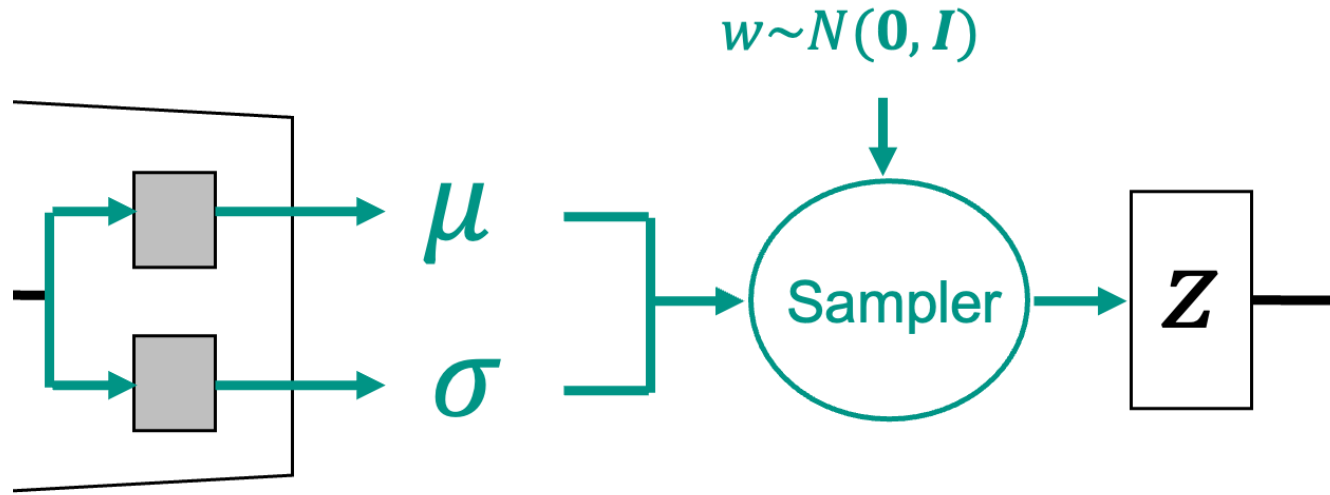


VAE



Add a probabilistic spin to AE: allow **sampling the latent space**

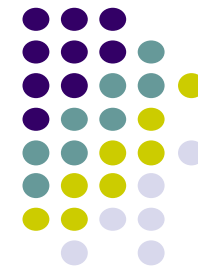
Variational AutoEncoders



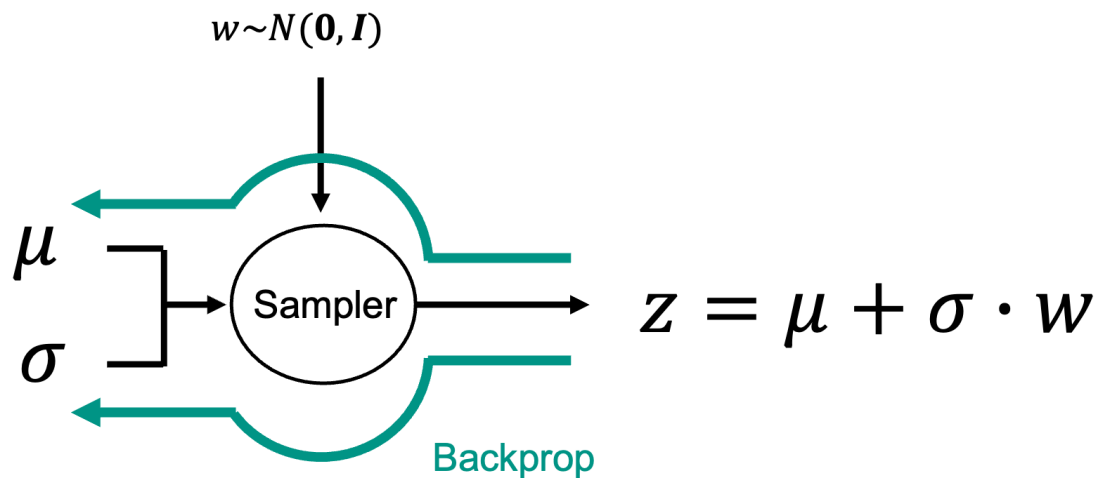
$q_{\theta}(\mathbf{z}) \approx p(\mathbf{z}|\mathbf{X})$: Proposal distribution

$p(\mathbf{z})$: Prior distribution

The Reparametrization trick



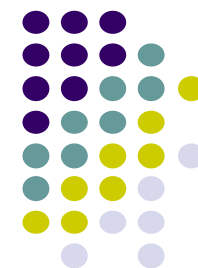
Way to sample a distribution whilst still being able to do backprop:



$$\frac{\partial z}{\partial \mu} = 1$$

$$\frac{\partial z}{\partial \sigma} = w$$

Variational AutoEncoders



Modification 2: loss function

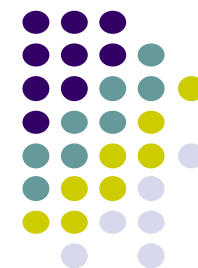
$$\mathcal{L}_r = \frac{1}{N_s} \sum_i \left(\mathcal{L}(\mathbf{x}^{(i)}, d_\phi(e_\theta(\mathbf{x}^{(i)}))) + KL(p(\mathbf{z}^{(i)}|\mathbf{x}^{(i)})||p(\mathbf{z}^{(i)})) \right)$$

↑↑
Reconstruction Loss (as in AEs) **Kullback-Leibler divergence**

Comes from solving this objective function:

$$\operatorname{argmin}_{\theta} KL(q_\theta(\mathbf{z})||p(\mathbf{z}|\mathbf{x}))$$

VAEs in practice



Learning Algorithm

1. Select a batch of training samples and feed to encoder
2. Divide the output of the encoder into two parts: mean and std (each of them has the same number of samples as the input batch)
3. Sample a latent space realization per training sample
4. Feed each latent vector into the decoder
5. Compute the loss: 1) MSE of reconstruction, 2) KL between prior and proposal using the mean and std obtained in step2. + backprop.

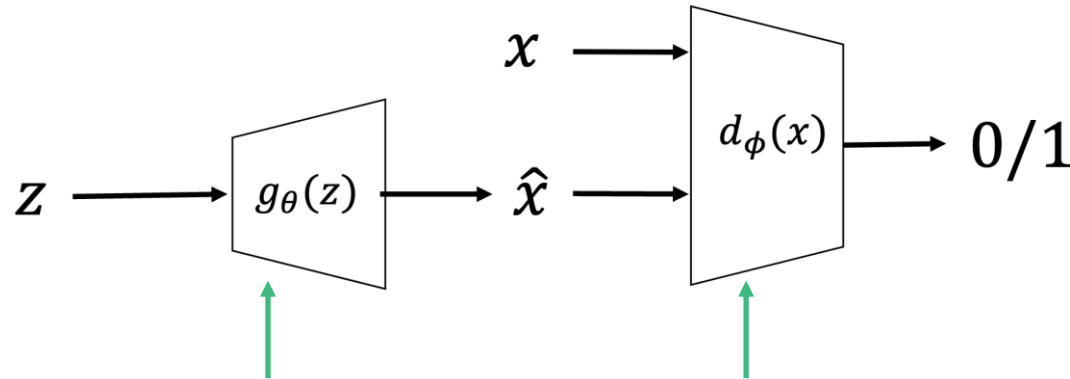
Sampling Algorithm

1. Sample a latent space realization
2. Feed each latent vector into the decoder

Generative Adversarial Networks



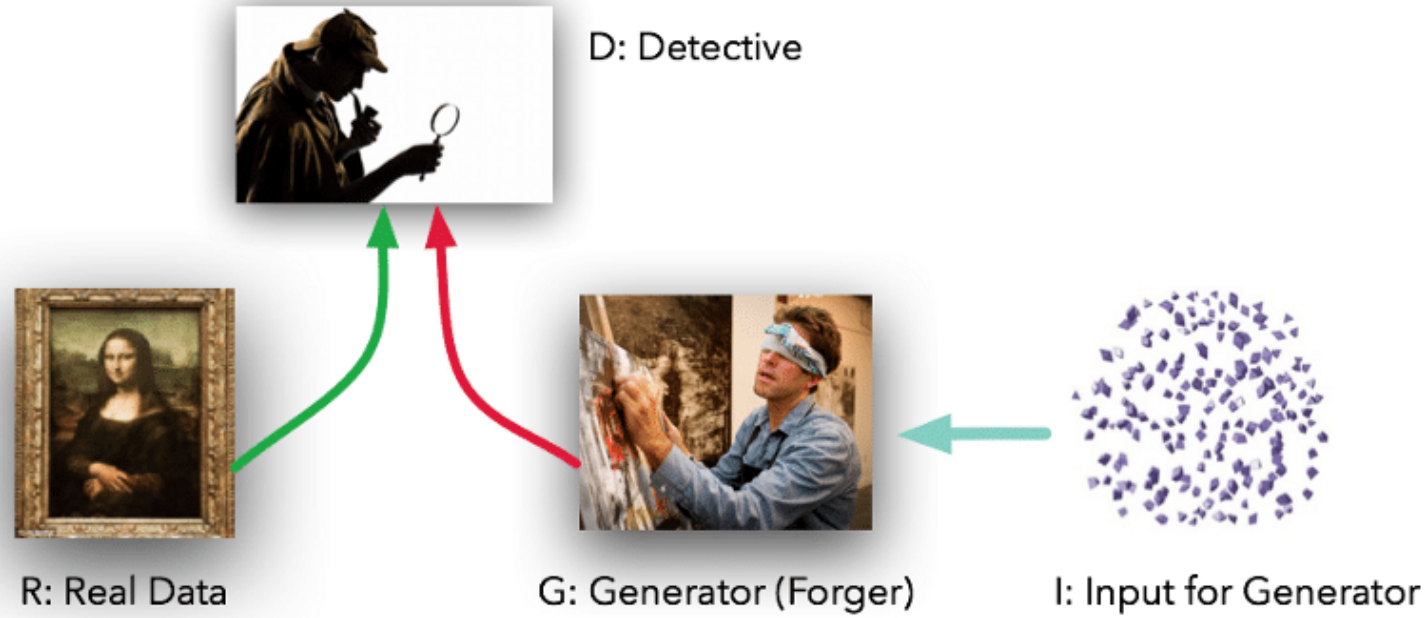
*Family of implicit generative models (do not explicitly estimate a density function), that are composed of two neural networks competing with each other – **adversarial training** (or 2-player min-max game)*



Generator: create a sample of the distribution of the training data from a noise realization z .

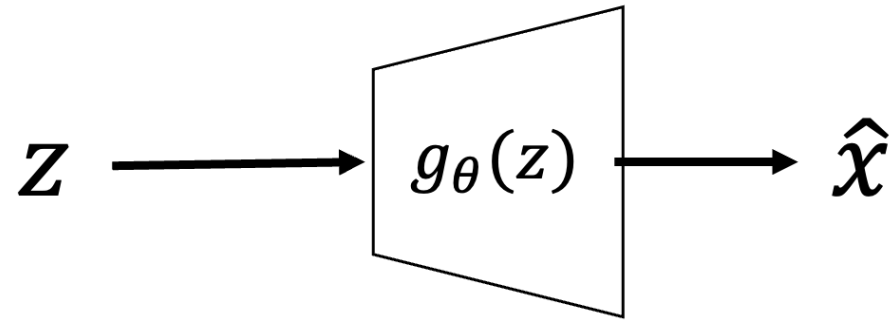
Discriminator: simple classifier tasked to tell if input is true or generated

Generative Adversarial Networks



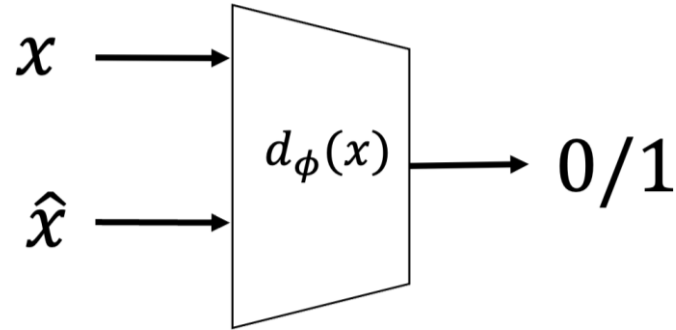
Source: <https://www.novatec-gmbh.de/en/blog/gans-applied-on-novatec-employees/>

GANs Generator



- $\mathbf{z} \in \mathbb{R}^{n_z}$ (with each element sampled from $\mathcal{N} \sim (0,1)$)
- $\hat{\mathbf{x}} \in \mathbb{R}^{n_x}$ created by feeding the vector $\mathbf{z}$ into the generator network
- $\mathbf{z}$ and $\hat{\mathbf{x}}$ can be 1-dimensional, 2-dimensional, or more. Sometimes $\mathbf{z}$ can be of the same dimension of $\hat{\mathbf{x}}$, but sometimes the network can bring a 1d array into a 2d array (common for images in the early days of GANs).
- **No direct training of generator**, gets an indirect feedback from discriminator.

GANs Discriminator

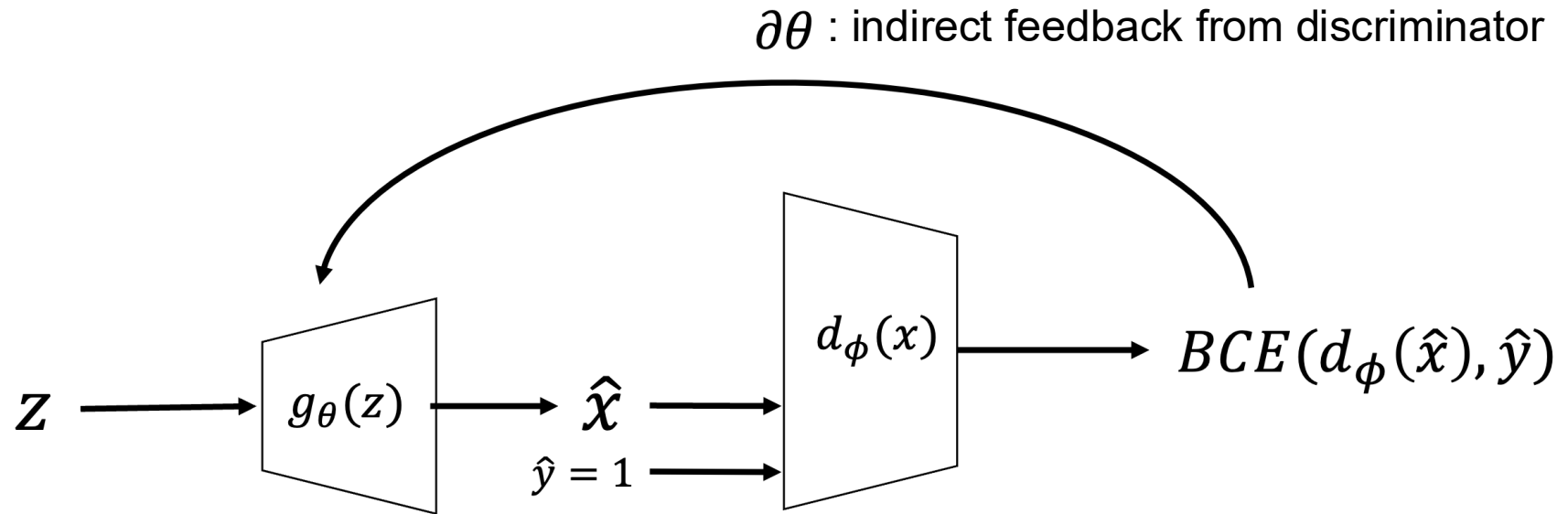


- $\hat{x} \in \mathbb{R}^{n_x}$ created by feeding the vector z into the generator network
- $x \in \mathbb{R}^{n_x}$ sample of the training dataset, from which we want to learn the underlying probability density function ($p(x)$).
- **Discriminator** is trained as **binary classifier** (i.e., BCE loss). Its goal is to tell apart fake ($\hat{x}$) from true (x) samples.

GANs Training process



Training is performed in two phases: **generator phase**

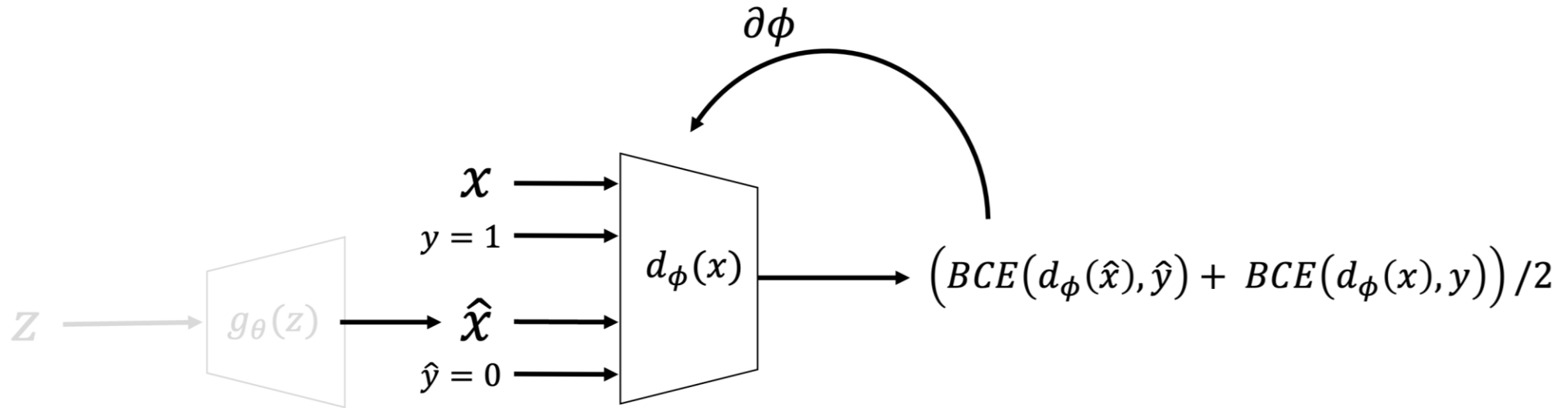


$$\mathcal{L}(\hat{\mathbf{x}}, \hat{\mathbf{y}} = 1) = BCE(d_\phi(g_\theta(\mathbf{z})), \hat{\mathbf{y}} = 1) = -\log(d_\phi(g_\theta(\mathbf{z})))$$

GANs Training process



Training is performed in two phases: **discriminator phase**



GANs Training process



The generator is trying to minimize the following loss function:

$$\mathcal{L}(\hat{\mathbf{x}}, \hat{\mathbf{y}} = 1) = BCE(d_{\phi}(g_{\theta}(\mathbf{z})), \hat{\mathbf{y}} = 1) = -\log(d_{\phi}(g_{\theta}(\mathbf{z})))$$



Min when discriminator is
fooled ($d_{\phi}(g_{\theta}(\mathbf{z})) = 1$)

The discriminator is trying to minimize the following sum of loss functions:

$$\mathcal{L}(\hat{\mathbf{x}}, \hat{\mathbf{y}} = 0) = BCE(d_{\phi}(g_{\theta}(\mathbf{z})), \hat{\mathbf{y}} = 0) = -\log(1 - d_{\phi}(g_{\theta}(\mathbf{z})))$$

$$\mathcal{L}(\mathbf{x}, \mathbf{y} = 1) = BCE(d_{\phi}(\mathbf{x}), \mathbf{y} = 1) = -\log(d_{\phi}(\mathbf{x}))$$



Min when
discriminator is good
($d_{\phi}(g_{\theta}(\mathbf{z})) = 0, d_{\phi}(\mathbf{x}) = 1$)

GANs in practice



Learning Algorithm

1. Select a batch of training samples and feed them through the discriminator
2. Create a batch of noise samples and feed them through the generator and discriminator
3. Compute the discriminator part of the loss function + backprop.
4. Repeat 1-2-3 N_{disc} times.
5. Create a batch of noise samples and feed them through the generator and discriminator and compute the generator part of the loss function + backprop.

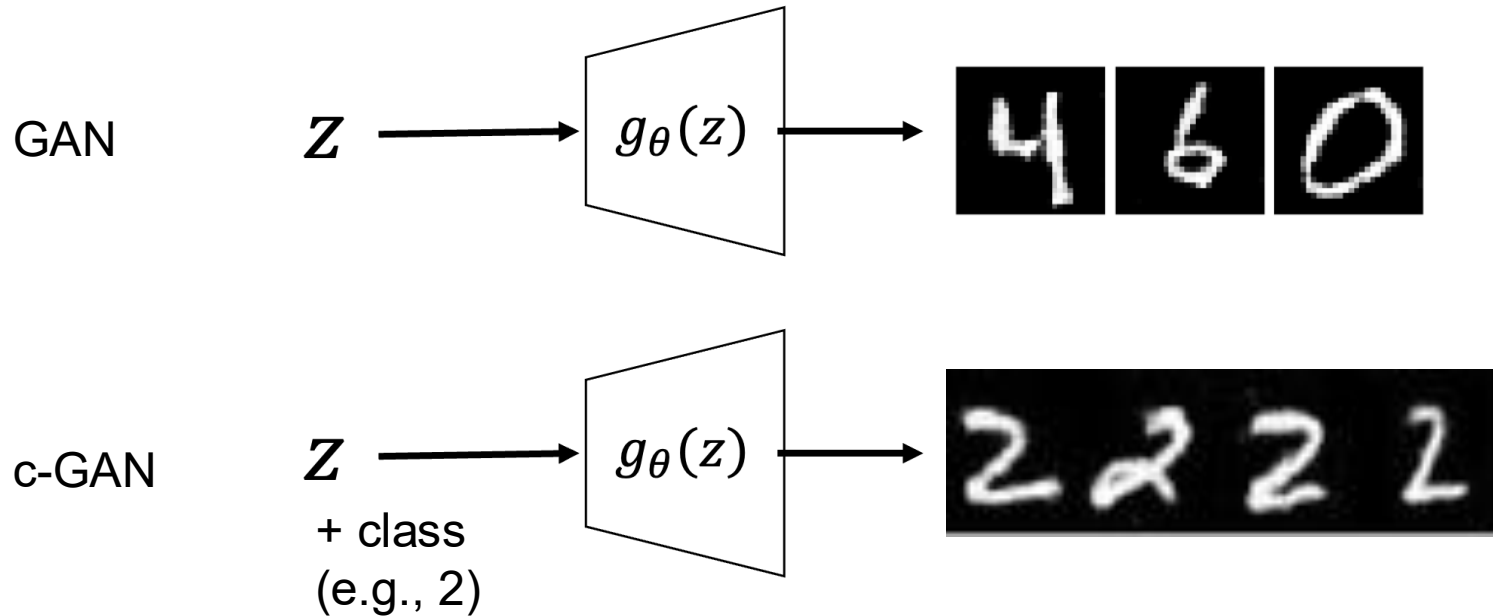
Sampling Algorithm

1. Create a noise sample
2. Feed it into the generator

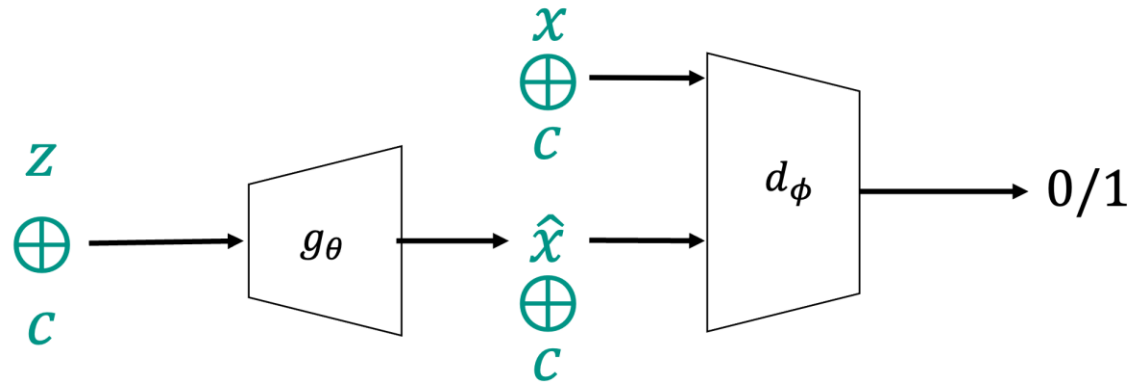
Conditional GANs



Idea: can we know upfront (before sampling) what we will produce...



Conditional GANs

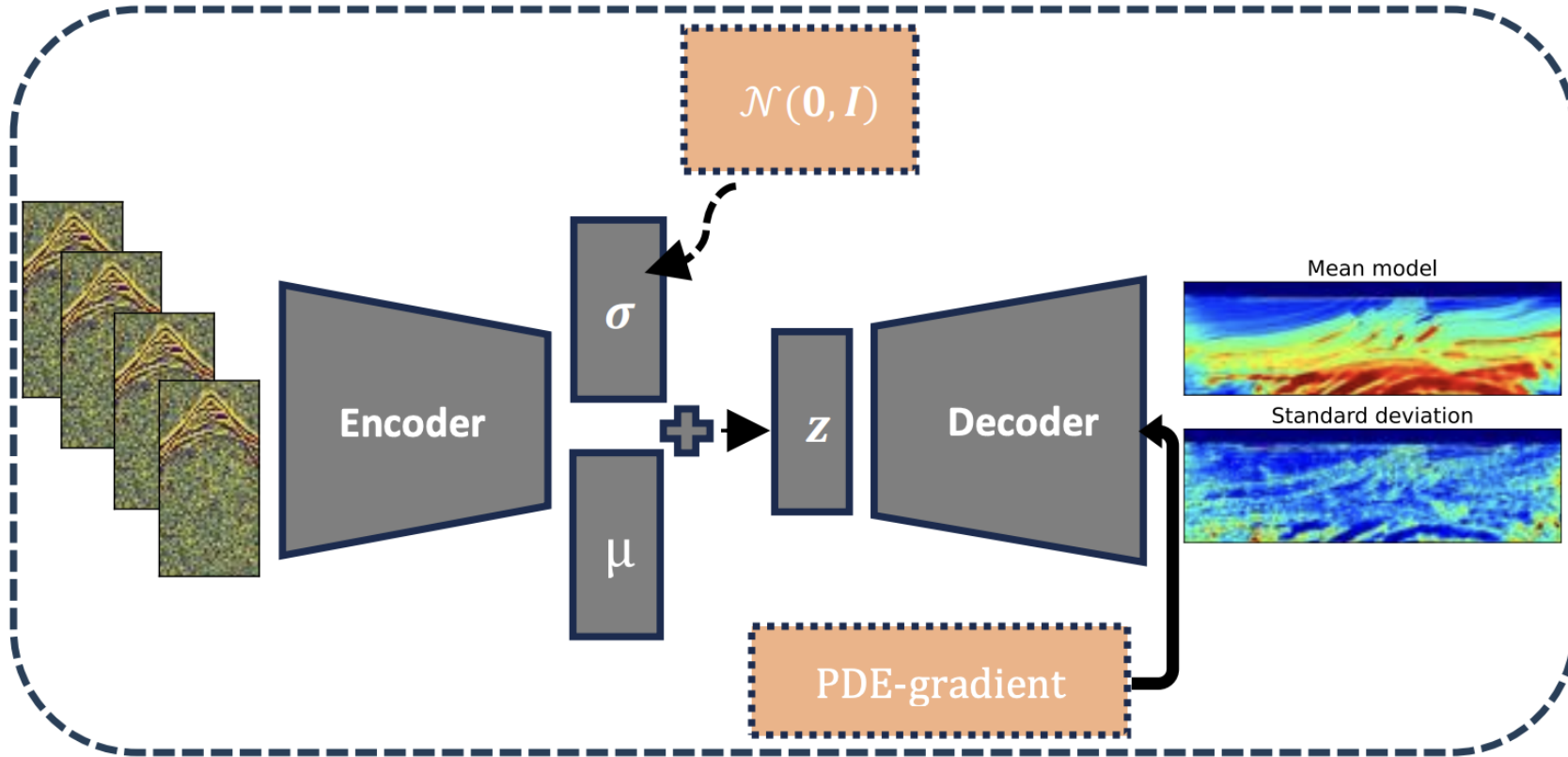


- Concatenate a label c representing a class to random noise ($z \oplus c$), or its one-hot encoded version ($z \oplus c$). Inform generator both at train and generation time.
- Concatenate the same label to x and $\hat{x}$ before feeding them to the discriminator.
- When working with Nd-arrays (e.g., images), concatenation is replaced by channel concatenation (c or c is replicated over the entire 'image').

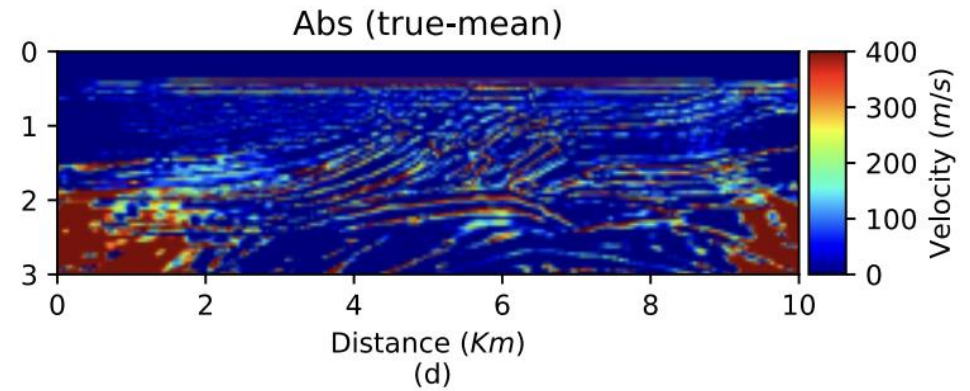
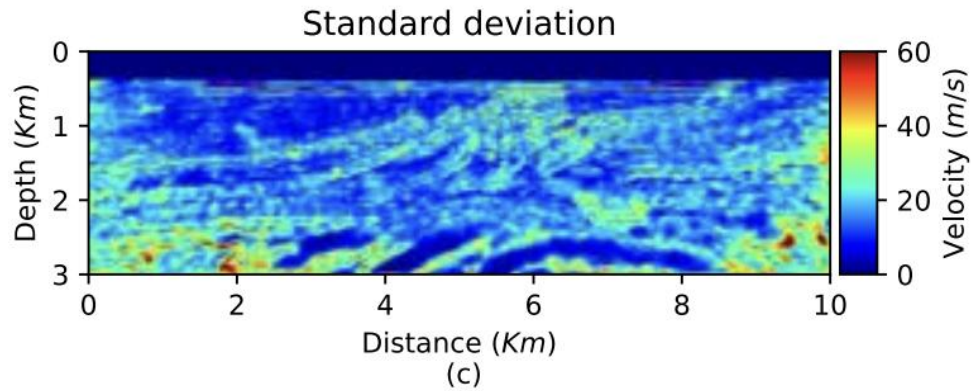
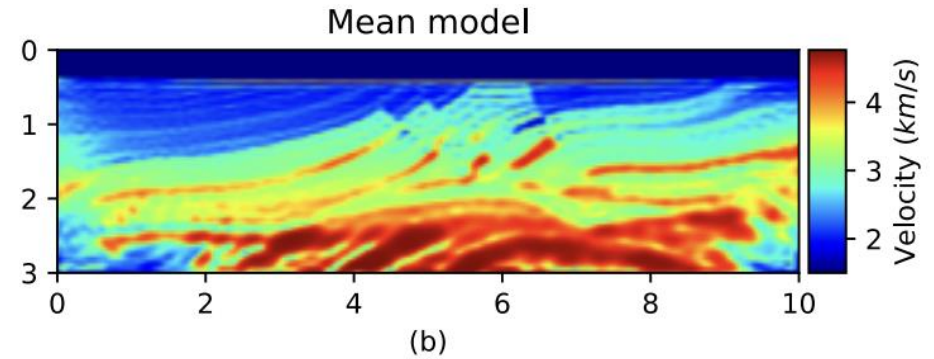
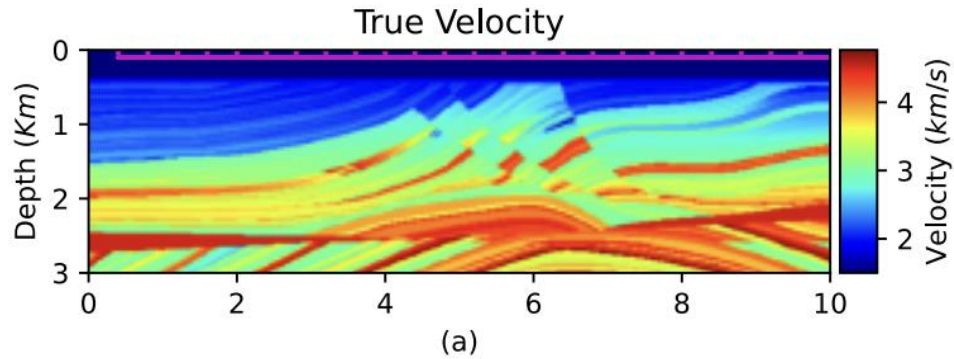




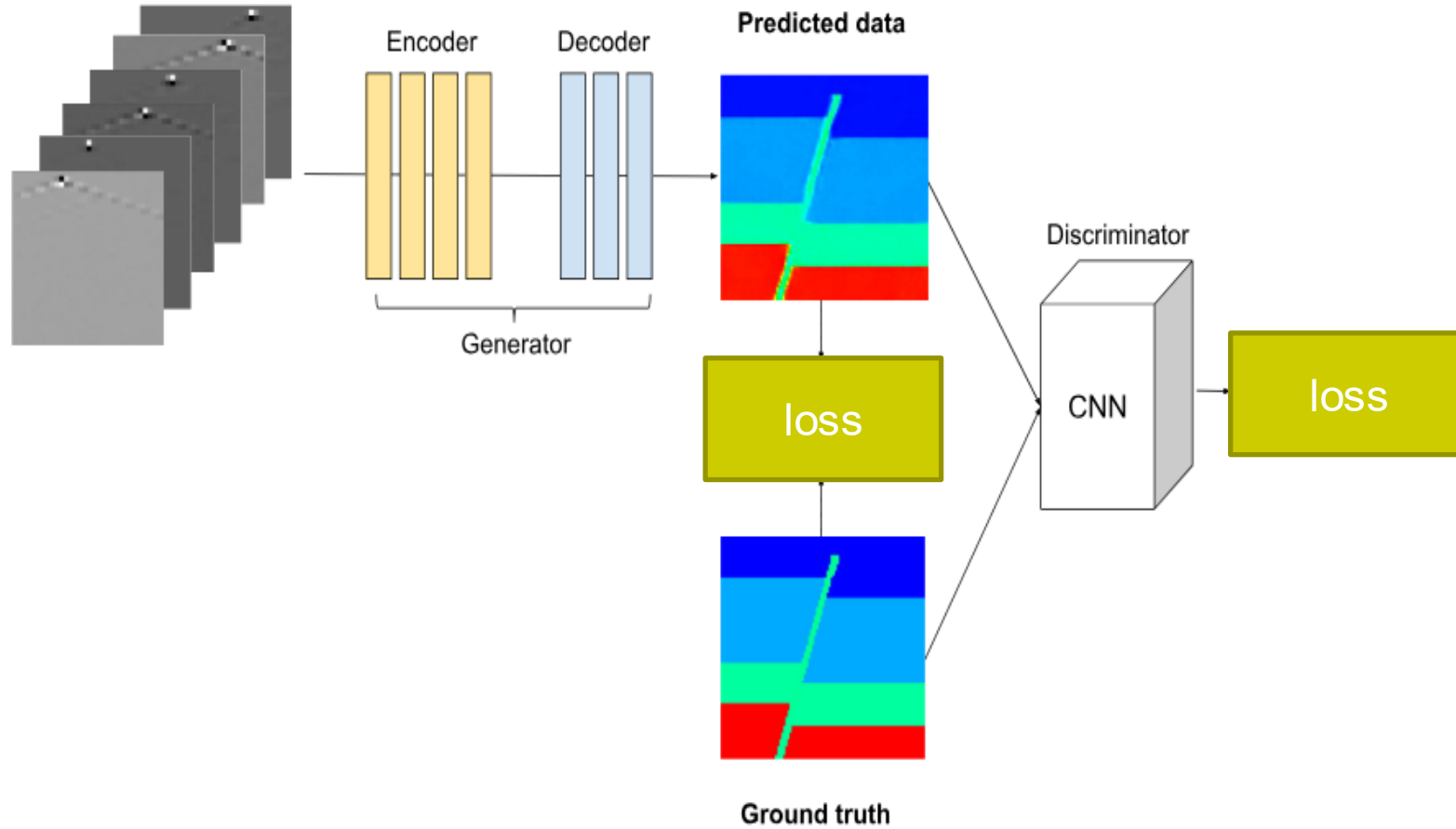
Applications



Uncertainty quantification of single and multi-parameter full-waveform inversion through a variational autoencoder



Uncertainty quantification of single and multi-parameter full-waveform inversion through a variational autoencoder



Zhang, Z., & Lin, Y. (2020). Data-driven seismic waveform inversion: A study on the robustness and generalization. *IEEE Transactions on Geoscience and Remote sensing*, 58(10), 6900-6913.